Give your AI agent the internet · 5 simple steps

AdiTejus · AI in simple steps — step card · free tools only · about 20 new lines

Before you start Video 1's folder `first-agent`: `agent.py`, `notes/`, and `.env` with the free Gemini key. Open it in VS Code, with its terminal. Mac: type `python3` wherever this card says `python`.

1 What "the web" means

FIND a page then **READ** a page — The answer is on the page.

2 A search tool of our own

In `agent.py`: four edits, then a free test.

1 At the very top, add two lines:

```
import urllib.parse
import urllib.request
```

2 Under `read_file`, add the function: 7 lines. Type them exactly, with the spaces at the start of each line.

```
def search_wikipedia(query):
    address = "https://en.wikipedia.org/w/api.php?" + urllib.parse.urlencode(
        {"action": "opensearch", "search": query, "limit": 3, "format": "json"})
    request = urllib.request.Request(address, headers={"User-Agent": "first-agent-tutorial/1.0"})
    reply = json.loads(urllib.request.urlopen(request, timeout=30).read())
    titles, links = reply[1], reply[3]
    return "\n".join(f"{t}: {u}" for t, u in zip(titles, links)) or "No results."
```

3 Tell the model it exists. In the `TOOLS` list, after the `read_file` entry, add one more entry: 5 lines.

```
    {"type": "function", "name": "search_wikipedia",
     "description": "Search Wikipedia. Returns up to three page titles with their web addresses.",
     "parameters": {"type": "object",
                    "properties": {"query": {"type": "string"}},
                    "required": ["query"]}},
```

4 Under the `TOOLS` list, add the `OURS` line. Then change the last line of `run_tool` to the `return` line.

```
OURS = {"list_files": list_files, "read_file": read_file, "search_wikipedia": search_wikipedia}
```

```
    return OURS[name](**args)
```

5 Test it alone: free, no request to the model. New file `test_search.py`, two lines. Save both files, then run it:

```
from agent import search_wikipedia
print(search_wikipedia("Python programming language"))
```

`python test_search.py` → a page title and its address.

3 The reading tool

Google runs it. We only switch it on.

In `TOOLS`, after the search entry, add one line: `{"type": "url_context"},` URL = a web address.

In `agent`, under the line `for turn in range(max_turns):`, add three lines. Mind the spaces at the start.

```
        for s in interaction.steps:
            if s.type == "url_context_call":
                print(f"   web page  -> {s.arguments}", flush=True)
```

4 A goal that needs the web

Free tier, my account: 20 requests a day. One run uses about 5.

At the bottom of `agent.py`, replace the old goal with this one:

```
    print(agent("What is the newest stable version of Python? Search Wikipedia, read the page "
                "you find, and answer in two sentences with the address of the page you read."))
```

Save, then run: `python agent.py` You should see a `tool call -> search_wikipedia` line, a `web page ->` line, then the answer, with the address it read.

5 Trust, but check

Open the address it gives. Compare with python.org/downloads. Then name the source:

```
    print(agent("What is the newest version of Python? Read https://www.python.org/downloads/ "
                "and answer in one sentence, with the address you read."))
```

If something goes wrong

Never click "Set up billing" unless you mean to pay.

`SyntaxError / IndentationError`	A typing slip on the line Python names. Compare it with the download.
`NameError / TypeError`	An edit from step 2 is missing. Go back and compare.
`URLError`	Check your internet, then run again.
`429`	Read the message. "per minute": wait a minute. "per day": come back tomorrow.
`Nothing for 5 minutes`	Press Ctrl+C. Usually the daily limit: come back tomorrow.

Public pages only · practise with public pages and sample notes

Code and README: link in the video description